

Intelligence Engine — Module Architecture

1. Overview

The Intelligence Engine is the core data-generation layer. It takes a product + company profile as input and produces structured export intelligence across 8 domains — market demand, buyers, competitors, variants, pricing, keywords, email sequences, and ad concepts.

Every module follows the same two-phase pattern:

Sonar (retrieval) → GPT (structuring)

- Perplexity Sonar is used for real-time factual retrieval only. Prompts are short (1-2 lines). Sonar fetches live web data and returns free-text.
- GPT-4o / GPT-4o-mini is used for structuring, classification, scoring, and writing. It never fetches data — it only processes what Sonar already returned.
- This division matters because Sonar is non-deterministic and can "deflect" (refuse to answer). GPT handles that gracefully.

2. Module Inventory

#	Module	File	LLM Calls	DB Table
1	Market Demand	modules/market_demand.py	1 Sonar + 1 GPT per country	product_info.market_intelligence
2	Buyer Discovery	modules/buyer_discovery.py	1 Sonar + 2 GPT	product_info.buyer_discovery
3	Competitor Discovery	modules/competitor_discovery.py	1 Sonar + 2 GPT	product_info.competitor_discovery
4	Variants & Formats	modules/variants_kit/variants_formats.py	2 Sonar + 4 GPT	product_info.variants_formats
5	Price Analysis	modules/price_analysis.py	2 Sonar + 3 GPT	product_info.price_analysis

6	Keyword Intelligence	modules/marketing_kit/keyword_intel.py	Google Ads API + 2 GPT	product_info.keyword_intelligence
7	Email Sequence	modules/marketing_kit/email_sequence.py	1 GPT	product_info.email_sequence
8	Ad Concepts	modules/marketing_kit/ad_concepts.py	1 GPT	product_info.ad_concepts

3. BaseModule — Shared Infrastructure

[File: modules/base_module.py](#)

All 8 modules extend BaseModule. It provides shared services so no module needs to manage API clients, retry logic, or extraction plumbing on its own.

3.1 ModuleInput

Standardised input object passed to every module. Built from product_master and company_preference tables.

Field	Type	Description
product_id	str	UUID
product_name	str	e.g. "Organic Turmeric Powder"
category	str	e.g. "spices"
hs_code	str	e.g. "0910.30"
certifications	list[str]	e.g. ["USDA Organic", "GMP"]
origin_country	str	Where the company is based
target_country	str	Which market to research

company_name	str	Used in ad concepts + competitor exclusion
business_type	str	Manufacturer / Exporter / Trader
price_positioning	str	Budget / Mid-range / Premium
moq	str	Minimum order quantity
buyer_type	str	B2B / B2C / Both

3.2 ModuleResult

Standardised output from Market Demand module runs. (Marketing kit modules have their own result types.)

Field	Type	Description
product_id	str	Passed through from input
target_country	str	Which country this result covers
module_name	str	e.g. "market_demand"
data	dict	Structured extracted data
raw_sonar	str	Raw Sonar response (stored for debugging)
success	bool	False if any critical step failed
error	str	Error message if success=False

field_count() returns how many of the 8 tracked fields have real non-null, non-empty values. Used to filter low-quality results.

3.3 Shared Helper Methods

- **_get_openai()** `_get_openai()` — Lazy OpenAI client. Created on first use. Uses OPENAI_API_KEY.

- **_get_sonar()** `_get_sonar()` — Perplexity Sonar client using the OpenAI SDK with `base_url=https://api.perplexity.ai`. Uses `PERPLEXITY_API_KEY`.
- **_call_sonar(query)** `_call_sonar(query)` — Calls Sonar with exponential backoff retry. Returns the text response or `None` on failure.
- **_call_sonar_with_deflection_retry()** `_call_sonar_with_deflection_retry(query, label)` — Calls `_call_sonar`, then checks for deflection. If deflected, retries once. Use this where deflection is a concern.
- **_extract_structured(prompt)** `_extract_structured(prompt)` — Calls gpt-4o-mini with `response_format: json_object`, `temp=0`. Returns a parsed dict or `None`.
- **_generate_analyst_note()** `_generate_analyst_note(inp, data)` — Calls gpt-4o (`temp=0.7`) to generate a 2-3 sentence analyst insight after structured extraction.

3.4 Sonar Deflection Detection

Sonar sometimes refuses to answer with phrases like "cannot provide", "not available", "consult a professional". This is detected by `_sonar_deflected(text)` which requires 2 or more deflection phrases to trigger (to avoid false positives on normal prose).

The 10 tracked deflection phrases:

"do not include", "not available", "consult", "cannot provide", "no specific", "would need to", "unable to find", "not found in", "search results do not", "no data"

When detected, `_call_sonar_with_deflection_retry` retries once. Deflection is non-deterministic — the same query on a retry often returns real data.

4. Module Deep Dives

4.1 Module 1 — Market Demand

[File: modules/market_demand.py](#)

Entry point: `MarketDemandModule().run_all_countries(inp)`

This is the only module that returns a list of `ModuleResult` (one per country) instead of a single result.

Flow

- Step 1 — Country discovery (1 Sonar + 1 GPT): Sonar asks which 15 countries are the largest importers. GPT parses the response into a list. Fallback: 9 default markets if discovery fails.

- Step 2 — Per-country research (1 Sonar + 1 GPT per country, max 3 concurrent): Sonar fetches import demand, volume, buyers, seasonality, channels, and certifications. GPT extracts 7 structured fields. Countries with fewer than 4/7 fields populated are filtered out.
- Step 3 — Analyst note (1 GPT per country): Uses extracted data, not raw Sonar — ensures specificity.

Output Fields

Field	Description
demand_growth	{value, period} — e.g. +18% YoY
import_volume	{value, unit, year} — e.g. 18400 MT, 2024
matched_buyers	{count} — estimated active buyer count
peak_procurement	{period} — best months to approach buyers
primary_channel	{channel} — dominant sales channel
cert_require	{certifications[]} — what buyers require
cert_gap	{status, detail} — competitor certification gaps
analysis_note	GPT-generated 2-3 sentence insight

Concurrency: Countries are researched with `asyncio.Semaphore(3)` — max 3 parallel Sonar calls to stay within rate limits.

DB: One row per product × country in `product_info.market_intelligence`.

4.2 Module 2 — Buyer Discovery

[File: `modules/buyer\_discovery.py`](#)

Entry point: `BuyerDiscoveryModule().run(inp)`

Finds the top 10 companies in `target_country` that actively use or source the product.

Flow (3 LLM calls)

- Sonar: "Who are the top 10 companies in {target_country} that use {product_name}?"
- GPT: Extracts structured list — name, website, buyer_type, notes
- GPT: Dedup, validate, remove vague entries, rank by outreach value

BuyerEntry fields: name, country, website (domain only), buyer_type (supplement_brand / food_manufacturer / retailer / cosmetics_brand / industrial / distributor / other), notes.

Quality threshold: is_fallback() returns True if fewer than 6 buyers were found — signals sparse data to the UI.

4.3 Module 3 — Competitor Discovery

[File: modules/competitor_discovery.py](#)

Entry point: CompetitorDiscoveryModule().run(inp)

Finds the top 10 global manufacturers and exporters competing with the user's product.

Flow (3 LLM calls)

- Sonar: "Who are the top 10 global manufacturers and exporters of {product_name}?"
- GPT: Extracts name, origin_country, website, competitor_type, notes. Filters OUT distributors, resellers, trading companies.
- GPT: Dedup, validate, rank by competitive threat to origin_country exporter

CompetitorEntry fields: name, origin_country, website, competitor_type (manufacturer / exporter / vertically_integrated), notes.

Downstream use: result.for_intel_module() returns the list in the format expected by competitor_intel.py — each entry includes product_name and product_id for enrichment.

4.4 Module 4 — Variants & Formats

[File: modules/variants_kit/variants_formats.py](#)

Prompts: modules/variants_kit/variants_prompts.py

Entry point: VariantsFormatsModule().run(inp)

Discovers trending product variants from real market data, then structures, tags, scores, and annotates them.

Why 2 Sonar calls instead of 1: A single Sonar call asking for both variant names AND full market data tends to return shallow data for each variant. Splitting into two calls — discover names first, then ask specific questions about that known list — returns significantly richer results.

Flow (6 LLM calls)

- Sonar call 1: "What are the 6 trending variants/formats of {product_name} in B2B export markets?"
- GPT call 1: Extracts clean list of variant names from Sonar free-text
- Sonar call 2: "What is the price, MOQ, demand, and lead time for [list of known variant names]?" (With deflection retry)
- GPT call 2: Structures 6 variants × 6 fields each
- No LLM: Fuzzy match to identify "your_product" from the variant list (SequenceMatcher ratio ≥ 0.2)
- GPT calls 3 + 4 (parallel): Tag/score (your_product / gap / in_market / emerging) + Analysis notes per variant

VariantData Fields

Field	Description
variant_name	e.g. "Turmeric Extract 95% Curcumin"
tag	your_product / gap / in_market / emerging
opportunity_score	1-10
key_spec	Primary differentiator (purity %, standardization)
price_range	e.g. "\$12-\$18/kg" — used by price_analysis
moq	Minimum order quantity
buyer_demand	High / Medium / Emerging / Low
matched_buyers	Estimated buyer count for this variant
lead_time	Typical delivery lead time
analysis_note	GPT-generated insight

4.5 Module 5 — Price Analysis

[File: modules/price_analysis.py](#)

Entry point: `PriceAnalysisModule().run(inp, variants_result)`

Depends on VariantsFormatsResult from Module 4 — variant names and price_range values are passed in, not re-fetched.

Flow

- Step 1 — Top metrics (1 Sonar + 1 GPT): FOB price range, gross margin %, cert premium. Extracts 3 stat cards: market_range, gross_margin, cert_premium_overall.
- Step 2 — Variant margin table (GPT only — no Sonar): GPT-4o-mini estimates margin % and market position for each variant using market_price from variants_formats + GPT knowledge of production costs.
- Step 3 — Cert premiums (1 Sonar + 1 GPT, parallel with Step 1): % premium that each certification commands vs uncertified commodity grade.

Position labels: Mid range / Uncontested / Opportunity / Easy upsell / Race to bottom / Premium

Position flags (UI hints): star / gap / neutral / upsell / warning / premium

4.6 Module 6 — Keyword Intelligence

[File: modules/marketing_kit/keyword_intel.py](#)

Entry point: KeywordIntelModule().run(inp)

The only module that calls an external non-LLM API — Google Ads KeywordPlanIdeaService.

Flow

- Step 1 — English keyword fetch (1 Ads API call): Seeds are product_name, supplier, bulk, manufacturer variants. All seeds passed in ONE combined request. Returns up to 200 keywords with real search volume + competition data.
- Step 2 — GPT classification + multilingual (parallel): Pre-filter B2B-trigger keywords + high-volume (≥ 500 /mo). GPT-4o-mini classifies into high_volume_buyer_intent and low_competition_gaps (max 10 each).
- Multilingual (non-English markets only): GPT generates 15 candidate keywords in target language. All 15 passed as seeds in ONE Ads API call. Match-back: exact → partial substring → unmatched (kept, volume=null).

B2B trigger words: supplier, wholesale, bulk, manufacturer, exporter, b2b, private label, oem, distributor, trade, import, certified, organic, factory, source, procurement

Rate limit handling: _ads_call_with_retry reads the "Retry in N seconds" value from the 429 error message. Falls back to 35 seconds if not parseable. Retries up to 3 times.

4.7 Module 7 — Email Sequence

[File: modules/marketing_kit/email_sequence.py](#)

Entry point: EmailSequenceModule().run(inp) — 1 × GPT-4o

Generates a 3-email B2B cold outreach sequence (cold intro, follow-up, value-add). No Sonar call — uses product data from input directly.

4.8 Module 8 — Ad Concepts

[File: modules/marketing_kit/ad_concepts.py](#)

Entry point: AdConceptsModule().run(inp) — 1 × GPT-4o (temp=0.9)

Generates 6 creative B2B ad concepts for LinkedIn ads, cold outreach, and email subject lines. Dynamic, not templated — GPT chooses the best 6 angles for the specific product and industry.

GPT selects from a broad menu: trust/proof-of-origin, urgency/supply window, spec-first, sustainability/ESG, category challenger, FOMO, cost-efficiency, niche specialization, regulatory readiness, OEM/partnership, volume reliability, reformulation trigger, brand-building, etc.

At least 1 concept must name the company (company_name) specifically.

AdConcept Fields

Field	Description
market	Target geography (e.g. "US MARKET", "EU MARKETS")
angle	Label in ALL CAPS (e.g. "TRUST ANGLE", "SPEC-FIRST")
hook	Ad headline, ≤12 words, punchy
description	2-3 sentences on why this hook works strategically
tiles	2-3 short tags (channel + buyer type)
border_color	UI hint: blue / orange / green / purple / teal / gray

5. Data Flow

ModuleInput

1

```
→ MarketDemandModule.run_all_countries()
```

```
|      ↳ list[ModuleResult]  (one per country)
```

1

```
└─ BuyerDiscoveryModule.run()
```

```
└─ BuyerDiscoveryResult
```

1

```
└─> CompetitorDiscoveryModule.run()
```

```
└─> CompetitorDiscoveryResult
```

1

```
|→ VariantsFormatsModule.run()
```

```
└─▶ VariantsFormatsResult
```

```
→ PriceAnalysisModule.run(inp, variants_result) ←
```

```
└─ PriceAnalysisResult
```

1

↳ Marketing Kit (run in parallel):

```
└─ KeywordIntelModule.run() → KeywordResult
```

```
└─ EmailSequenceModule.run() → EmailSequenceResult
```

```
└─> AdConceptsModule.run()      → AdConceptsResult
```

PriceAnalysisModule is the only module that depends on another module's output.

All others take ModuleInput directly.

6. Configuration Reference

[File: input_pipeline/config.py](#)

Config Key	Used By	Value / Notes
LLM["sonar_model"]	BaseModule._call_sonar	
LLM["extraction_model"]	BaseModule._extract_structured (gpt-4o-mini)	
LLM["generation_model"]	BaseModule._generate_analyst_note (gpt-4o)	
LLM["max_retries"]	Both Sonar + GPT retry loops	
LLM["retry_delay_sec"]	Exponential backoff base	
MARKETING_KIT["ad_model"]	AdConceptsModule (gpt-4o)	
MARKETING_KIT["ad_temp"]	AdConceptsModule temperature	0.9
GOOGLE_ADS["yaml_path"]	KeywordIntelModule — Ads client credentials	
GOOGLE_ADS["customer_id"]	KeywordIntelModule — Ads account	

Environment variables required: **OPENAI_API_KEY**, **PERPLEXITY_API_KEY**

7. Error Handling Policy

All modules follow a never-raise contract. Every run() method catches all exceptions and returns a result object with success=False and an error string. The caller (module_runner) always gets a usable object.

Scenario	Behaviour
Sonar returns None after max retries	success=False, error="Sonar call failed after retries"
Sonar deflects twice	Second attempt returned as-is (may still have partial data)
GPT extraction returns invalid JSON	success=False, error="Structured extraction failed"
Ads API 429 rate limit	Retry with wait parsed from error message (default 35s)
Ads API fails entirely	KeywordResult(success=False) — module does not crash
Fewer than 5 buyers/competitors found	is_fallback=True flagged in result — UI can show warning
Market demand country has <4/7 fields	Country filtered out of run_all_countries results

8. Adding a New Module

- Create modules/your_module.py
- class YourModule(BaseModule):
- For simple 1-Sonar + 1-extract flows, implement: module_name(), build_query(inp), build_extraction_prompt(inp, sonar_response), empty_result(). Inherit run() unchanged.
- For multi-step flows: override run() and call inherited helpers (_call_sonar_with_deflection_retry, _extract_structured, _generate_analyst_note) directly.
- Add the module to module_runner.py and register its result serializer in intelligence_api.py.

Sonar prompt guideline: Keep it a direct factual question, 1-2 lines maximum. Sonar performs retrieval — it does not need context about why you are asking or what format you want. Long prompts hurt retrieval quality.